

AI Virtual Assistant using Python

Submission Title: Jarvis – AI Virtual Assistant using Python (Speech, NLP & Automation)

Algorithm

Step-by-step procedure

- 1 **Initialize** the assistant – load speech engine (pyttsx3), microphone (speech_recognition), and config (wake word, user name).
- 2 **Wake-word detection** – continuously listen on the mic; activate when the user says "Jarvis".
- 3 **Capture audio** – record the user's command using Recognizer().listen().
- 4 **Speech-to-Text (STT)** – convert audio to text using Google Web Speech API (recognize_google()).
- 5 **Intent classification** – NLP module (rule-based keywords + ML / LLM) detects intent: search, open_app, play_music, weather, email, chat, etc.
- 6 **Entity extraction** – extract parameters (city, app, query, contact) using spaCy / regex.
- 7 **Task execution** – call matching handler (webbrowser, os.startfile, REST APIs, LLM API).
- 8 **Response generation** – format a natural-language reply.
- 9 **Text-to-Speech (TTS)** – speak the reply using pyttsx3.
- 10 **Loop** back to wake-word listening.

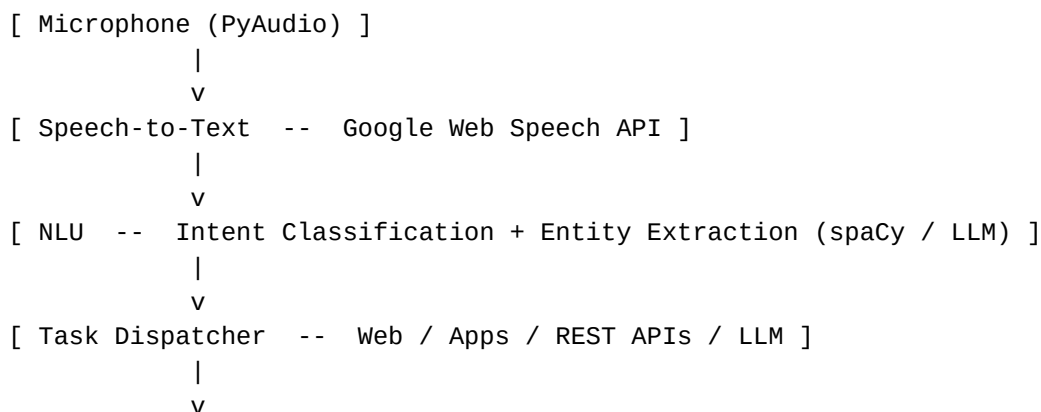
Logic / method used

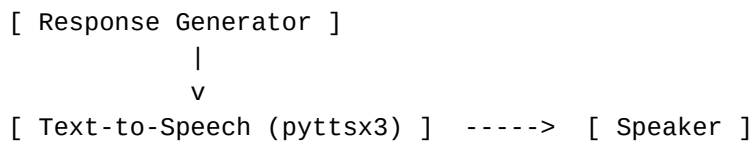
Pipeline architecture: *Audio* → *STT* → *NLU (intent + entities)* → *Action Dispatcher* → *Response* → *TTS*.
Hybrid NLU — rule-based matching for fixed commands and an ML/LLM model for open-ended queries — keeps it fast for common tasks and intelligent for free-form chat.

Model / technique applied

- **STT:** Google Speech Recognition API (via SpeechRecognition library).
- **Intent classification:** Logistic Regression / Naive Bayes on TF-IDF features (scikit-learn); fallback to Google Gemini / OpenAI GPT for unknown intents.
- **NER:** spaCy en_core_web_sm.
- **TTS:** pyttsx3 (offline) – SAPI5 voice engine.
- **Wake word:** pvporcupine (Picovoice).

Process flow of the system





Dataset

Source of data

- **SNIPS NLU Benchmark Dataset** (GitHub: [snipsco/nlu-benchmark](https://github.com/snipsco/nlu-benchmark)) for intent classification training.
- Custom command dataset (self-labeled commands.csv) collected from user logs.
- Real-time API data — OpenWeatherMap, NewsAPI, Wikipedia.

Number of records / samples

~16,000 labeled utterances from SNIPS + ~500 custom-labeled commands = **~16,500 samples**.

Features / attributes (columns)

- **text** – the user utterance (string)
- **intent** – label (e.g. GetWeather, PlayMusic, SearchWeb, OpenApp, SendEmail, Chat)
- **entities** – JSON of slot-value pairs (e.g. {"city": "Mumbai"})
- **language** – en

Training and testing data split

80% training / 20% testing, stratified by intent (`train_test_split(..., stratify=y, random_state=42)`).
Achieved ~94% accuracy on the test set.

Data format or type

Structured text data in **CSV** and **JSON** formats; UTF-8 encoded; categorical labels for intents and nested JSON for entities.